

# TA Course | Shader 进阶系列

## 第 2 节：Blinn-Phong 镜面反射模型

授课目标：在 Lambert 漫反射基础上加上高光，实现塑料、陶瓷等常见材质的真实质感。

### 2.1 为什么需要镜面反射（Specular）？

Lambert 漫反射只能做出哑光效果，而现实中很多物体都有闪亮的高光（如塑料、陶瓷、金属、光滑漆面）。

镜面反射的物理本质：  
光线在表面发生相对规则的反射，形成我们看到的高光亮点。高光的位置和形状会随着观察角度变化而移动。

### 2.2 Phong 模型 vs Blinn-Phong 模型

传统 Phong 模型：

- 计算反射向量  $R = \text{reflect}(-L, N)$
- 高光强度  $= (R \cdot V)^{\text{shininess}}$

缺点：计算量较大，且在某些角度会出现问题。

Blinn-Phong 模型（目前主流）：由 Jim Blinn 提出，使用半向量（Half Vector）代替反射向量，性能更好，效果接近。

核心公式：

$$\text{Specular} = (N \cdot H)^{\text{shininess}}$$

其中：

- $H = \text{normalize}(L + V)$  // 半向量 = 光方向 + 视线方向的归一化结果
- $L$  = 光线方向
- $V$  = 视线方向（从表面指向摄像机）
- shininess = 光滑度（数值越大，高光越小越亮）

为什么用半向量？

半向量  $H$  位于光方向和视线方向中间，当  $H$  与法线  $N$  越接近时，高光越强。这比计算反射向量更高效。

### 2.3 Blinn-Phong 视觉特点

- 高光是圆形或椭圆形的亮斑
- 高光颜色通常接近光源颜色（非金属）或物体颜色（金属）
- 光滑度越高，高光越集中、越刺眼
- 粗糙度越高，高光越柔和、范围越大

### 2.4 Blinn-Phong 模型的优缺点

优点：

- 计算速度快（比 Phong 更快）
- 效果自然，广泛用于游戏和实时渲染
- 可通过调整光滑度（Smoothness）轻松控制高光大小

缺点：

- 仍然是经验模型（非物理真实）
- 不满足能量守恒（高光可能过亮）
- 金属与非金属的高光颜色无法自动区分

第 2 节完整案例 Shader

```
````hsl Shader "TA_Course/Lesson2_BlinnPhong" { Properties { _BaseColor ("基础颜色", Color) = (1,1,1,1) _Smoothness ("光滑度", Range(0.0, 1.0)) = 0.5 } SubShader { Pass { Name "FORWARD" Tags { "LightMode"="UniversalForward" }
```

```

HLSLPROGRAM
#pragma vertex vert
#pragma fragment frag

#include "Packages/com.unity.render-pipelines.universal/ShaderLibrary/Core.hlsl"
#include "Packages/com.unity.render-pipelines.universal/ShaderLibrary/Lighting.hlsl"

struct appdata
{
    float4 vertex : POSITION;
    float3 normal : NORMAL;
};

struct v2f
{
    float4 pos : SV_POSITION;
    float3 normalWS : TEXCOORD0;
    float3 positionWS : TEXCOORD1;
};

float4 _BaseColor;
float _Smoothness;

v2f vert (appdata v)
{
    v2f o;
    VertexPositionInputs posInput = GetVertexPositionInputs(v.vertex.xyz);
    VertexNormalInputs normalInput = GetVertexNormalInputs(v.normal);

    o.pos = posInput.positionCS;
    o.normalWS = normalInput.normalWS;
    o.positionWS = posInput.positionWS;
    return o;
}

half4 frag (v2f i) : SV_Target
{
    float3 N = normalize(i.normalWS);
    float3 V = GetWorldSpaceNormalizeViewDir(i.positionWS);
    Light light = GetMainLight();

    float3 L = light.direction;
    float3 H = normalize(L + V); // 半向量

    // Lambert 漫反射
    float ndotl = saturate(dot(N, L));
    half3 diffuse = _BaseColor.rgb * light.color * ndotl;

    // Blinn-Phong 镜面反射
    float ndoth = saturate(dot(N, H));
    float specular = pow(ndoth, _Smoothness * 128.0);

    half3 finalColor = diffuse + specular * light.color;

    return half4(finalColor, 1.0);
}

ENDHLSL
}
}

```